

Module 7: Distributed File Systems

7.1 Architecture of Distributed File Systems

A Distributed File System (DFS) bridges geographical network gaps, allowing users and applications to access files stored remotely as if they were sitting on their local hard drive. To achieve this illusion, systems rely heavily on complex client-side caching mechanisms, dedicated server-side storage arrays, and a cohesive directory service.

Caching is central to DFS performance, and different systems make different choices about when a written change is actually made durable and visible to others:

Caching Strategy	Behavior	Trade-off
Write-through	Every write is immediately sent to the server.	Safe, but slower — every write pays network latency.
Write-back	Writes are held locally and flushed to the server later, in batches.	Fast, but risks data loss if the client crashes before flushing.

7.2 Network File System (NFS)

Developed by Sun Microsystems, NFS is the ubiquitous standard for distributed file access. It leverages RPC (see Module 1) for almost all of its network interactions — every file read or write is, under the hood, an RPC call to the file server.

Version	Statefulness	Notable Feature
NFSv2 / NFSv3	Stateless — the server keeps no record of open files or client status.	Simple, and server crash recovery is trivial since there is no session state to rebuild.
NFSv4	Stateful — the server tracks open files and client sessions.	Adds proper file locking and aggressive performance caching, at the cost of more complex crash recovery.

The word "stateless" here is doing a lot of work: because an early NFS server remembered nothing about which files a client had open, if the server crashed and rebooted, it could simply start serving requests again immediately — there was no session to lose. The trade-off was that features like proper file locking (which inherently requires remembering who is holding what) were difficult to support until NFSv4 introduced statefulness.

7.3 The Coda File System

Coda represents a leap forward, focusing heavily on mobility and high availability. Unlike NFS, Coda allows for **disconnected operation**: a user can cache files locally (a process called **hoarding**, where the system pre-loads the files it predicts you will need), physically disconnect from the network, continue to read and modify those files, and the Coda system will automatically synchronize the changes to the central servers upon reconnection.

The tricky part is **conflict resolution**: what happens if the same file was modified both on the disconnected laptop and on the server (perhaps by a different device) while the two were apart? Coda attempts automatic conflict resolution for simple cases, and flags genuinely conflicting changes for the user to resolve manually.

■ Disconnected Operation in Everyday Life

This is the same idea behind modern tools like Google Docs offline mode or Git. You keep working locally while offline (on a plane, for example), and when you reconnect, your changes are merged back in — automatically if possible, or flagged for you to review if two changes genuinely conflict.

■ Key Takeaway

Every DFS design is a balancing act between making remote files feel local (via caching) and keeping those cached copies honest and up to date (via statefulness, locking, or conflict resolution) — NFS and Coda simply sit at different points on that spectrum.

Glossary of Key Terms

Term	Meaning
DFS	Distributed File System — remote files accessed as if local.
Write-back cache	A cache that delays writing changes to the server until later.
Stateless server	A server that keeps no memory of client sessions between requests.
Hoarding	Coda's strategy of pre-caching files a user is likely to need offline.
Disconnected operation	Continuing to work on cached files while off the network.

What You Should Know

- The core architectural illusion a DFS provides to end-users, and the write-through vs write-back caching trade-off.
- The evolution of NFS from a stateless architecture (NFSv2/v3) to a stateful protocol (NFSv4), and what that trade-off buys and costs.
- How Coda achieves high availability through hoarding, aggressive caching, and disconnected operation support, including how it handles conflicts.